

HelixSpecifier

HelixSpecifier

작업량에 맞춰 자체 프로세스를 조정하는 스펙 주도 개발

요약

HelixSpecifier는 세 가지 개발 방법론—SpecKit의 스펙 주도 워크플로, Superpowers의 TDD 원칙, GSD의 마일스톤 라이프사이클—을 하나의 적응형 프로세스로 융합한 Go 엔진입니다. 각 작업을 노력도에 따라 분류하고 그에 맞춰 프로세스 규모를 유동적으로 조정합니다.

간략 설명

HelixSpecifier는 AI 에이전트를 위한 스펙 주도 개발 융합 엔진입니다. SpecKit, Superpowers, GSD를 결합하고 작업을 노력도에 따라 분류하며, 토론 기반 스펙 단계, 최소 테스트-코드 비율 강제, 완료된 모든 프로세스로부터 학습하는 기능을 제공합니다.

상세 설명

HelixSpecifier는 Go(모듈 `digital.vasic.helixspecifier`)로 작성된 스펙 주도 개발(SDD) 융합 엔진으로, HelixAgent AI 시스템의 구성 요소입니다. 일반적으로 세 가지 별도의 도구와 사고방식으로 분리되어 있던 세 가지 개발 방식을 하나의 적응형 워크플로로 통합합니다. 즉, SpecKit의 7단계 SDD 프로세스(Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), Superpowers의 테스트 주도 원칙과 병렬 서브 에이전트 실행, GSD의 마일스톤 라이프사이클 관리입니다. 각 기법은 본래의 강점을 유지하며, 엔진은 이들을 개별적으로 연결된 파이프라인이 아닌 하나의 일관된 프로세스로 운영합니다.

핵심 개념은 적응형 프로세스입니다. 엔진은 작업의 노력도를 분류하고 그에 맞춰 프로세스 규모를 조정하여, 간단한 한 줄 수정 작업이 대규모 기능 개발과 동일한 복잡한 절차를 거치지 않도록 하고, 반대로 대규모 기능이 사소한 오타 수정처럼 간단히 처리되지 않도록 합니다. 이 기본 구조를 바탕으로 열 가지 핵심 기능이 구현되어 있습니다. 동시성 제한 병렬 작업 실행, 자동 규칙 강제를 위한 기계 판독 가능한 “Constitution as Code”, 최소 테

스트-구현 비율을 추적하고 강제하는 “Nyquist TDD”, 스펙 정제를 위한 다중 라운드 다중 에이전트 토론, 적응형 역량 학습, 레거시 코드 분석, 과거 패턴 기반 예측 스펙, 프로젝트 간 지식 이전, 실행 중 프로세스 재조정, 의미 기반 검색이 가능한 지속적 스펙 메모리입니다.

HelixSpecifier는 Go 모듈로 제공되며(go get 또는 로컬 replace 지시어를 통해), 의도적으로 간결하게 설계된 엔진 API 뒤에서 동작합니다. 세 가지 기법과 프로세스 조정기, 스펙 메모리를 등록하고 작업의 노력도를 분류한 후 전체 프로세스를 실행하면 품질 점수가 부여된 결과를 얻을 수 있습니다. 표면적으로는 단순하지만, 그 이면의 오케스트레이션은 복잡합니다. Helix 제품군과 마찬가지로, 모의 테스트가 아닌 실제 코드를 활용한 인프로세스 챌린지 러너를 통해 엄격한 검증 체계를 갖추고 개발되었습니다.

개발 배경

스펙 주도 개발, 엄격한 TDD, 마일스톤 관리는 보통 세 가지 별도의 도구와 방식으로 운영됩니다. HelixSpecifier는 AI 에이전트(HelixAgent)가 이 세 가지를 하나의 일관되고 자동으로 확장되는 워크플로우로 실행할 수 있도록 개발되었습니다. 수작업으로 연결하는 대신, 자체적으로 통합된 프로세스를 제공합니다.

왜 혁신적인가

이 시스템은 프로세스를 자동으로 작업량에 비례하게 조정합니다. 팀은 보통 두 가지 극단에 갇히곤 합니다. 모든 일에 복잡한 절차를 적용해 안전하지만 느리고 은근히 반감을 사는 방식이거나, 절차 없이 빠르게 진행하다가 결국 문제가 터지는 방식입니다. HelixSpecifier는 이러한 딜레마를 해소합니다. 각 작업의 분류된 노력도에 맞춰 절차의 규모를 조정하고, 작업이 진행되면서 실시간으로 재조정합니다. 이전에는 실현 불가능했던 기능, 즉 작업별로 최적화된 프로세스와 더불어 다중 라운드, 다중 에이전트 토론을 통한 입장 점수 평가로 결정되는 명세화 방식이 바로 그것입니다.

혁신적인 요소

- **적응형 절차** - 프로세스 수준이 사전 고정치 아닌 실시간 품질 지표에 따라 런타임에 조정됩니다.
- **나이퀴스트 TDD** - 테스트 대 구현 비율 게이트(최소 2배)를 적용하며, 나이퀴스트 샘플링 정리의 논리를 차용합니다. 행동을 정확하게 포착하려면 그 속도보다 훨씬 높은 빈도로 샘플링해야 하므로, 테스트는 커버하는 코드보다 더 많은 측정이 필요합니다.
- **토론 아키텍처** - 단일 의견을 대체하는 다중 라운드, 다중 에이전트 명세 개선 방식으로, 제안된 입장이 점수화되고 수렴됩니다.
- **예측형 명세화 및 프로젝트 간 지식 이전** - 엔진이 축적된 흐름을 분석해 명세를 예측하고, 한 프로젝트에서 얻은 귀중한 지식을 다음 프로젝트로 이전합니다.

- 코드로 표현된 **Constitution** - 프로젝트 규칙을 기계가 읽을 수 있는 형태로 변환하고 엔진이 강제 실행하도록 하여 검토자의 주의력에 의존하지 않습니다.

가장 큰 기술적 도전 과제와 해결 방법

- 서로 충돌하지 않는 세 가지 방법론의 통합 - SpecKit, Superpowers, GSD는 각각 워크플로우를 독립적으로 가정합니다. 이를 해결하기 위해 공통 인터페이스 뒤에 각 기동을 등록하고 하나의 공유 흐름 라이프사이클로 구동하는 융합 엔진을 개발했습니다. 이로써 세 가지 방법론이 충돌하지 않고 하나의 프로세스로 통합됩니다.
- 특정 작업에 필요한 프로세스 양의 결정 - 과도하게 추정하면 모든 것이 느려지고, 과소평가하면 위험한 작업이 검증 없이 배포됩니다. 이를 해결하기 위해 작업량을 분류하는 노력도 분류기를 도입하고, 실행 과정에서 동적으로 프로세스 수준을 조정하는 절차 스케일러를 적용했습니다.
- 인간 게이트키퍼 없이 명세 품질 유지 - 단발성 명세를 토론 기반 개선 방식으로 대체하고, 에이전트가 여러 라운드에 걸쳐 경쟁 입장을 점수화하도록 했습니다. 또한 나이퀴스트 TDD 비율을 강제해 구현이 테스트를 앞설 수 없도록 했습니다.

기술 스택

- **Go** - 엔진을 단일 임포트 가능한 바이너리로 제공하기 위해 선택되었으며, 런타임 부담이 없습니다. 이 시스템의 동시성 모델 덕분에 제한된 병렬 작업 디스패치와 다중 에이전트 토론 라운드가 스레딩 문제 없이 실현 가능합니다.
- **logrus** - 엔진과 세 가지 기동 전반에 걸쳐 구조화된 로깅을 제공해 흐름의 결정(분류, 절차 변경, 토론 결과 등)이 사후에도 명확히 파악될 수 있도록 합니다.
- **SpecKit** 기동 - 7단계 명세 주도 개발 프로세스(Constitution → 명세화 → 명확화 → 계획 → 작업 → 분석 → 구현)를 제공해 명세가 코드로 변환되는 체계적인 기반을 마련합니다.
- **Superpowers** 기동 - TDD 원칙을 준수하며 병렬 하위 에이전트 실행을 통해 테스트 우선의 엄격함과 구현의 신속성을 보장합니다.
- **GSD** 기동 - 마일스톤 및 라이프사이클 관리를 통해 흐름에 '완료'의 개념과 단계별 진행을 부여합니다.
- 명세 메모리 저장소 - 과거 명세의 지속적이며 의미론적으로 검색 가능한 인덱스로, 예측형 명세화와 프로젝트 간 지식 이전의 기반이 되어 매번 처음부터 시작하지 않아도 됩니다.

상태 및 정직성 안내

- 상태: 베타. HelixAgent의 Go 모듈 구성 요소로 사용됨.
- 라이선스: 미정. GitHub API를 통해 라이선스가 감지되지 않음 — 미검증 / 미선언.
- 표시 이름 "HelixSpecifier"은 저장소 specifier에 매핑됨.

우선 순위 단계: Helix-주요.